

Лабораторная работа №7

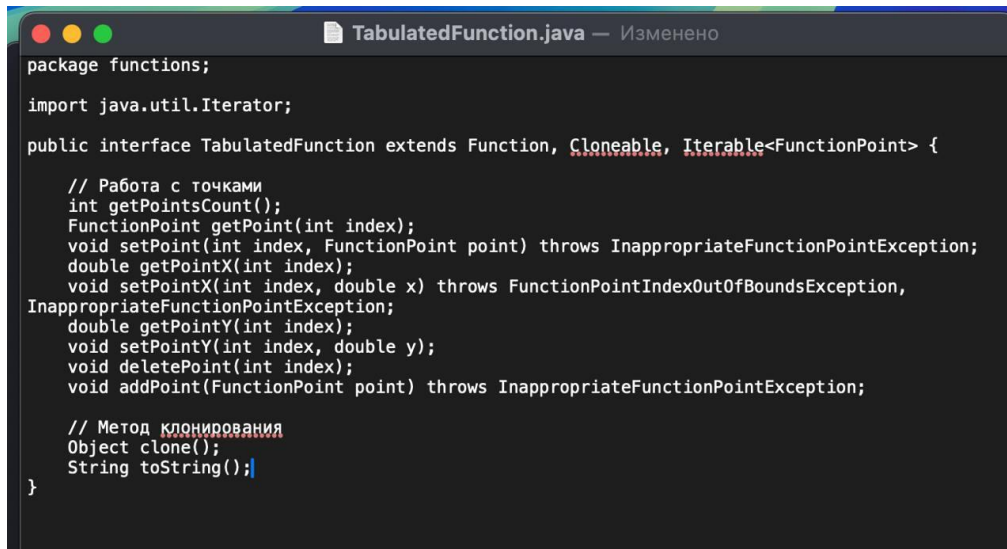
Валиневич Владислав Александрович

группа: 6204-010302D

Цель работы: Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).

Задание 1:

Добавим интерфейс `Iterable<FunctionPoint>`, чтобы использовать объекты `TabulatedFunction` в цикле `for-each`.

A screenshot of a code editor window titled "TabulatedFunction.java — Изменено". The code defines a package "functions" and imports "java.util.Iterator". It then defines a public interface "TabulatedFunction" that extends "Function", "Cloneable", and "Iterable<FunctionPoint>". The interface includes several methods: "getPointsCount()", "getPoint(int index)", "setPoint(int index, FunctionPoint point)", "getPointX(int index)", "setPointX(int index, double x)", "getPointY(int index)", "setPointY(int index, double y)", "deletePoint(int index)", "addPoint(FunctionPoint point)", "clone()", and "toString()".

```
package functions;

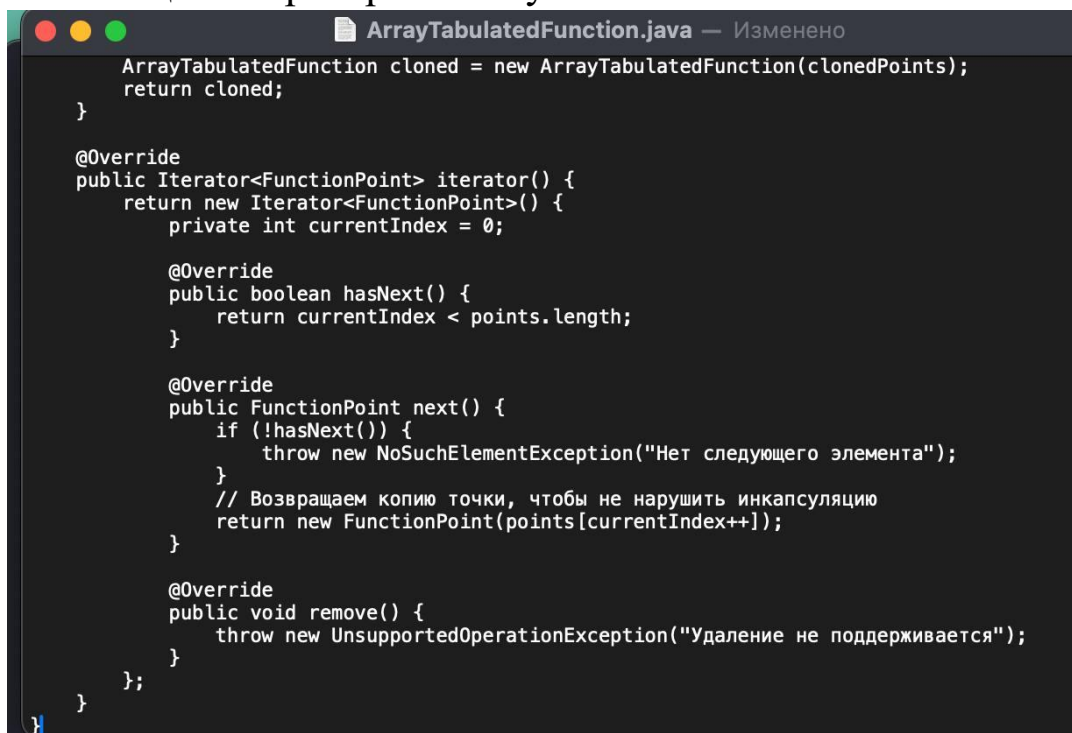
import java.util.Iterator;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> {

    // Работа с точками
    int getPointsCount();
    FunctionPoint getPoint(int index);
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;
    double getPointX(int index);
    void setPointX(int index, double x) throws FunctionPointIndexOutOfBoundsException,
    InappropriateFunctionPointException;
    double getPointY(int index);
    void setPointY(int index, double y);
    void deletePoint(int index);
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;

    // Метод клонирования
    Object clone();
    String toString();
}
```

Реализация итератора в ArrayTabulatedFunction:

A screenshot of a code editor window titled "ArrayTabulatedFunction.java — Изменено". The code shows a class "ArrayTabulatedFunction" with a constructor that takes "clonedPoints" and returns "cloned". It implements the "Iterable<FunctionPoint>" interface by overriding "iterator()", "hasNext()", "next()", and "remove()". The "iterator()" method returns a new "Iterator<FunctionPoint>()" with a private "currentIndex" set to 0. The "hasNext()" method returns "currentIndex < points.length". The "next()" method checks if "hasNext()" is true, throws a "NoSuchElementException" if not, and then returns a new "FunctionPoint" object with "points[currentIndex++]". The "remove()" method throws a "UnsupportedOperationException" with the message "Удаление не поддерживается".

```
ArrayTabulatedFunction cloned = new ArrayTabulatedFunction(clonedPoints);
return cloned;
}

@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < points.length;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            // Возвращаем копию точки, чтобы не нарушить инкапсуляцию
            return new FunctionPoint(points[currentIndex++]);
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
```

Реализация итератора в LinkedListTabulatedFunction:

```
LinkedListTabulatedFunction.java — Изменено

@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.next;
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < pointsCount;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            FunctionPoint result = new FunctionPoint(currentNode.point);
            currentNode = currentNode.next;
            currentIndex++;
            return result;
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException("Удаление не поддерживается");
        }
    };
}
```

Итераторы напрямую работают с внутренними структурами данных:

В `ArrayTabulatedFunction` — с массивом `points`

В `LinkedListTabulatedFunction` — с узлами связного списка

Протестируем в `main`:

```
public class main {
    public static void main(String[] args) {

        System.out.println("==== Задание 1 =====");
        // Тестирование ArrayTabulatedFunction
        System.out.println("\n=== Тестирование ArrayTabulatedFunction ===");
        FunctionPoint[] points = {
            new FunctionPoint(0, 0),
            new FunctionPoint(1, 1),
            new FunctionPoint(2, 4),
            new FunctionPoint(3, 9),
            new FunctionPoint(4, 16)
        };

        TabulatedFunction arrayFunc = new ArrayTabulatedFunction(points);
        System.out.println("Использование for-each цикла:");
        for (FunctionPoint p : arrayFunc) {
            System.out.println(p);
        }

        System.out.println("\n=== Тестирование LinkedListTabulatedFunction ===");
        TabulatedFunction linkedListFunc = new LinkedListTabulatedFunction(points);
        System.out.println("Использование for-each цикла:");
        for (FunctionPoint p : linkedListFunc) {
            System.out.println(p);
        }
    }
}
```

```
лаб7 — -zsh — 80x18

===== Задание 1 =====

=== Тестирование ArrayTabulatedFunction ===
Использование for-each цикла:
(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)

=== Тестирование LinkedListTabulatedFunction ===
Использование for-each цикла:
(0.0; 0.0)
(1.0; 1.0)
(2.0; 4.0)
(3.0; 9.0)
(4.0; 16.0)
vladislavvalinevich@MacBook-Air-Vladislav лаб7 %
```

Задание 2:

Реализуем интерфейс TabulatedFunctionFactory:

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Реализуем фабрики в классах:

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

```

public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}

```

TabulatedFunctions:

```

package functions;

import java.io.*;

public class TabulatedFunctions {

    // Приватный конструктор, чтобы нельзя было создать объект класса
    private TabulatedFunctions() {}

    // Фабрика по умолчанию (создает ArrayTabulatedFunction)
    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory factory) {
        TabulatedFunctions.factory = factory;
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return factory.createTabulatedFunction(leftX, rightX, values);
    }

    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return factory.createTabulatedFunction(points);
    }

    // Табулирует функцию на заданном отрезке с заданным количеством точек
    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) {
        // Проверяем, что границы табулирования находятся в области определения функции
        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
            throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
        }

        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек должно быть не менее 2");
        }

        if (leftX >= rightX) {
            throw new IllegalArgumentException("Левая граница должна быть меньше правой");
        }

        // Создаем массив значений Y
        double[] values = new double[pointsCount];
        double step = (rightX - leftX) / (pointsCount - 1);

        // Заполняем массив значений, вычисляя функцию в каждой точке
        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            values[i] = function.getFunctionValue(x);
        }

        // Возвращаем табулированную функцию с помощью фабрики
        return createTabulatedFunction(leftX, rightX, values);
    }

    // Выводит табулированную функцию в байтовый поток
    public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) {
        try (DataOutputStream dataout = new DataOutputStream(out)) {
            dataout.writeInt(function.getPointsCount());

```

Протестируем в main:


```

System.out.println("==== Задание 2 =====");
// Тестирование фабрик
System.out.println("=== Тестирование фабричного метода ===\n");

Function f = new Cos();
TabulatedFunction tf;

// Изначально используем фабрику по умолчанию (ArrayTabulatedFunction)
tf = TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println("Фабрика по умолчанию: " + tf.getClass());

// Меняем на фабрику LinkedListTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println("После установки LinkedListTabulatedFunctionFactory: " + tf.getClass());

// Меняем обратно на фабрику ArrayTabulatedFunction
TabulatedFunctions.setTabulatedFunctionFactory(new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());
tf = TabulatedFunctions.tabulate(f, 0, Math.PI, 11);
System.out.println("После установки ArrayTabulatedFunctionFactory: " + tf.getClass());

}
}

==== Задание 2 =====
=== Тестирование фабричного метода ===

Фабрика по умолчанию: class functions.ArrayTabulatedFunction
После установки LinkedListTabulatedFunctionFactory: class functions.LinkedListTabulatedFunction
После установки ArrayTabulatedFunctionFactory: class functions.ArrayTabulatedFunction

```

Задание 3:

Добавим методы в TabulatedFunctions:

```

public static TabulatedFunction createTabulatedFunction(
    Class<?> clazz, double leftX, double rightX, int pointsCount) {

    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(clazz)) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не реализует интерфейс TabulatedFunction");
    }

    try {

        Constructor<?> constructor = clazz.getConstructor(double.class, double.class, int.class);

        // Создаем экземпляр с помощью найденного конструктора
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, pointsCount);

    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не имеет конструктора (double, double, int)", e);
    } catch (Exception e) {
        throw new IllegalArgumentException(
            "Ошибка при создании объекта класса " + clazz.getName(), e);
    }

}

public static TabulatedFunction createTabulatedFunction(
    Class<?> clazz, double leftX, double rightX, double[] values) {

    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(clazz)) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не реализует интерфейс TabulatedFunction");
    }

    try {

        Constructor<?> constructor = clazz.getConstructor(double.class, double.class, double[].class);

        // Создаем экземпляр с помощью найденного конструктора
        return (TabulatedFunction) constructor.newInstance(leftX, rightX, values);

    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не имеет конструктора (double, double, double[])", e);
    } catch (Exception e) {
        throw new IllegalArgumentException(
            "Ошибка при создании объекта класса " + clazz.getName(), e);
    }

}

```

```

public static TabulatedFunction createTabulatedFunction(
    Class<?> clazz, FunctionPoint[] points) {

    // Проверяем, что класс реализует интерфейс TabulatedFunction
    if (!TabulatedFunction.class.isAssignableFrom(clazz)) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не реализует интерфейс TabulatedFunction");
    }

    try {
        // Находим конструктор с параметрами (FunctionPoint[])
        Constructor<?> constructor = clazz.getConstructor(FunctionPoint[].class);

        // Создаем экземпляр с помощью найденного конструктора
        return (TabulatedFunction) constructor.newInstance((Object) points);
    } catch (NoSuchMethodException e) {
        throw new IllegalArgumentException(
            "Класс " + clazz.getName() + " не имеет конструктора (FunctionPoint[])", e);
    } catch (Exception e) {
        throw new IllegalArgumentException(
            "Ошибка при создании объекта класса " + clazz.getName(), e);
    }
}

```

Проверка интерфейса: Метод `isAssignableFrom()` проверяет, реализует ли класс интерфейс `TabulatedFunction`.

Поиск конструктора: Используется `getConstructor()` для поиска конструктора с нужными параметрами.

Обработка исключений: Все исключения рефлексии преобразуются в `IllegalArgumentException` с сохранением оригинальной причины.

Протестируем `main`:

```

System.out.println("=== Задание 3 ===\n");

TabulatedFunction ff;

// Создание ArrayTabulatedFunction с помощью рефлексии
System.out.println("Создание ArrayTabulatedFunction через границы и количество точек:");
ff = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, 3);
System.out.println("Класс: " + ff.getClass());
System.out.println("Функция: " + ff);

// Создание ArrayTabulatedFunction с помощью рефлексии (массив значений)
System.out.println("\nСоздание ArrayTabulatedFunction через границы и массив значений:");
ff = TabulatedFunctions.createTabulatedFunction(ArrayTabulatedFunction.class, 0, 10, new double[] {0, 10});
System.out.println("Класс: " + ff.getClass());
System.out.println("Функция: " + ff);

//Создание LinkedListTabulatedFunction с помощью рефлексии (массив точек)
System.out.println("\nСоздание LinkedListTabulatedFunction через массив точек:");
ff = TabulatedFunctions.createTabulatedFunction(
    LinkedListTabulatedFunction.class,
    new FunctionPoint[] {
        new FunctionPoint(0, 0),
        new FunctionPoint(10, 10)
    }
);
System.out.println("Класс: " + ff.getClass());
System.out.println("Функция: " + ff);

//Табулирование функции Sin с помощью рефлексии (создание LinkedListTabulatedFunction)
System.out.println("\nТабулирование функции Sin с созданием LinkedListTabulatedFunction:");
ff = TabulatedFunctions.tabulate(
    LinkedListTabulatedFunction.class, new Sin(), 0, Math.PI, 11);
System.out.println("Класс: " + ff.getClass());
System.out.println("Функция: " + ff);
}

```

```

=== Задание 3 ===

Создание ArrayTabulatedFunction через границы и количество точек:
Класс: class functions.ArrayTabulatedFunction
Функция: {(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}

Создание ArrayTabulatedFunction через границы и массив значений:
Класс: class functions.ArrayTabulatedFunction
Функция: {(0.0; 0.0), (10.0; 10.0)}

Создание LinkedListTabulatedFunction через массив точек:
Класс: class functions.LinkedListTabulatedFunction
Функция: {(0.0; 0.0), (10.0; 10.0)}

Табулирование функции Sin с созданием LinkedListTabulatedFunction:
Класс: class functions.LinkedListTabulatedFunction
Функция: {(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.2566370614359172; 0.9510565162951535), (1.57079632679
48966; 1.0), (1.884955921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.224
6467991473532E-16)}

```

Вывод: Я внес изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).